SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:		**Reg. No.:**	
Section / Batch:		**Date:**	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- C(machine) = 3
- C(machine learning) = 3
- C(learning improves) = 1
- C(learning enables) = 1
- C(learning drives) = 1
- C(learning improves business) = 1

The interpolation weights are:

- λ_1 = 0.5 (Trigram)
- λ_2 = 0.3 (Bigram)
- λ_3 = 0.2 (Unigram)

The prediction probabilities after the word **"learning"** are:

- P(improves) = 0.33
- P(enables) = 0.33
- P(drives) = 0.33

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **P(learning | machine)**. Interpret how this probability can influence the next-word prediction of an e-commerce search system.
2. A customer enters the unseen sequence **"machine learning transforms"**. Apply the Backoff Model and estimate the probability of the word **"transforms"** using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.
3. Using Deleted Interpolation, calculate the probability of the sequence **"machine learning improves"** using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **P(improves) = 0.33, P(enables) = 0.33, P(drives) = 0.33**. Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.

5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. **"Book an appointment with the doctor."**
2. **"The book contains medical information."**

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word **"book"**, the system is given:

- P(book | VB) = 0.7
- P(book | NN) = 0.3
- P(Start → VB) = 0.6
- P(Start → NN) = 0.4

The system must determine whether **"book"** functions as a Verb (VB) or Noun (NN) depending on its context.

Questions

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to **"book"** using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of **"book"** being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.
3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.
4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection, entity identification, and response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

"Market growth drives investment."

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

Questions

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for **"drives"**. Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.
4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule **"Change NNS to VBZ if the preceding word is tagged as NN."** The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

Given Corpus

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The three sentences are:

1. machine learning improves business
2. machine learning enables automation
3. machine learning drives innovation

Important counts

- C(machine) = 3
- C(learning) = 3
- C(machine learning) = 3
- C(learning improves) = 1
- C(learning enables) = 1
- C(learning drives) = 1
- C(learning improves business) = 1

Total number of words:

Q1. Calculate MLE of Bigram Probability P(learning | machine)

The MLE formula for a bigram is:

This means that in the training corpus, **every occurrence of "machine" is followed by "learning"**.

Therefore, when a customer types:

machine

the search system will strongly predict:

learning

This can improve autocomplete because the system recognizes the highly frequent phrase **"machine learning"**.

Q2. Backoff Model for "machine learning transforms"

The customer enters:

machine learning transforms

The word **"transforms"** does not occur anywhere in the training corpus.

We first try the trigram:

Since:

the trigram probability is:

The backoff model then moves to the bigram:

But learning transforms also does not occur.

Therefore:

Finally, we consider the unigram probability:

Since transforms never occurs:

Hence:

Therefore, using the given unsmoothed corpus:

Why is Backoff useful?

Backoff is useful because a search query may contain combinations that were **not present in the training corpus**.

The model works from:

Trigram → Bigram → Unigram

For example:

P(word | previous 2 words)

↓ if unavailable

P(word | previous 1 word)

↓ if unavailable

P(word)

In a real e-commerce system, smoothing or an <UNK> token would normally be used so that completely unseen

words can receive a small non-zero probability.

Q3. Deleted Interpolation
We use:
Given:
For:
machine learning improves
Trigram probability
Bigram probability
Unigram probability
improves occurs once.
Total words = 12.
Therefore:
Interpolated probability
If the supplied probability 0.33 is used for both trigram and bigram:
Using exact :
So the exact result is:
Why interpolation is useful
Interpolation combines different levels of information:

- **Trigram** → uses two previous words
- **Bigram** → uses one previous word
- **Unigram** → uses overall word frequency

This makes the model more robust when training data is sparse.

Q4. Entropy
Given:
Entropy is:
Therefore:
Because the given probabilities sum to 0.99 due to rounding, using the exact equal distribution gives:
Interpretation
The three words have almost equal probabilities.
Therefore, the system is **highly uncertain** about which word will come next.
For three equally likely choices, entropy is maximum:
Thus, the search system has significant uncertainty when predicting the next word after **learning**.

Q5. Python Program – Case Study 1

```
import nltk

    if unigram_count > 0:
        p_backoff = unigram_count / len(tokens)
    else:
        p_backoff = 0

    print("Unigram probability =", p_backoff)

print("P(transforms | machine learning) =", p_backoff)

# ----------------------------------------------------
# Q3: DELETED INTERPOLATION
# ----------------------------------------------------

print("\n--- DELETED INTERPOLATION ---")

lambda1 = 0.5
lambda2 = 0.3
lambda3 = 0.2
```

```
# Trigram
p_tri = trigram_counts[
    ("machine", "learning", "improves")
] / bigram_counts[("machine", "learning")]

# Bigram
p_bi = bigram_counts[
    ("learning", "improves")
] / unigram_counts["learning"]

# Unigram
p_uni = unigram_counts["improves"] / len(tokens)

print("Trigram probability =", p_tri)
print("Bigram probability =", p_bi)
print("Unigram probability =", p_uni)

interpolated_probability = (
    lambda1 * p_tri
    + lambda2 * p_bi
    + lambda3 * p_uni
)

print("\nInterpolation:")
print("0.5 ×", p_tri)
print("+ 0.3 ×", p_bi)
print("+ 0.2 ×", p_uni)

print("Final probability =",
      interpolated_probability)

# ----------------------------------------------------
# Q4: ENTROPY
# ----------------------------------------------------

print("\n--- ENTROPY ---")

prediction_probs = np.array([0.33, 0.33, 0.33])

entropy = 0

for p in prediction_probs:
    entropy -= p * math.log2(p)

print("Probabilities:", prediction_probs)
print("Entropy =", entropy, "bits")

# ----------------------------------------------------
# FINAL PREDICTION
# ----------------------------------------------------

words = ["improves", "enables", "drives"]

prediction = words[np.argmax(prediction_probs)]
```

```
print("\n--- FINAL NEXT-WORD PREDICTION ---")
print("Predicted word:", prediction)

print("\n" + "=" * 60)
```

```
--- BIGRAM COUNTS ---
('machine', 'learning') : 3
('learning', 'improves') : 1
('improves', 'business') : 1
('business', ',') : 1
(',', 'machine') : 2
('learning', 'enables') : 1
('enables', 'automation') : 1
('automation', ',') : 1
('learning', 'drives') : 1
('drives', 'innovation') : 1

--- TRIGRAM COUNTS ---
('machine', 'learning', 'improves') : 1
('learning', 'improves', 'business') : 1
('improves', 'business', ',') : 1
('business', ',', 'machine') : 1
(',', 'machine', 'learning') : 2
('machine', 'learning', 'enables') : 1
('learning', 'enables', 'automation') : 1
('enables', 'automation', ',') : 1
('automation', ',', 'machine') : 1
('machine', 'learning', 'drives') : 1
('learning', 'drives', 'innovation') : 1

--- MLE BIGRAM PROBABILITY ---
C(machine) = 3
C(machine, learning) = 3
P(learning | machine) = 1.0

--- BACKOFF MODEL ---
Trigram unseen -> Backing off to Bigram
Bigram unseen -> Backing off to Unigram
Unigram probability = 0
P(transforms | machine learning) = 0

--- DELETED INTERPOLATION ---
Trigram probability = 0.3333333333333333
Bigram probability = 0.3333333333333333
Unigram probability = 0.07142857142857142
```

```
Interpolation:
0.5 × 0.3333333333333333
+ 0.3 × 0.3333333333333333
+ 0.2 × 0.07142857142857142
Final probability = 0.28095238095238095

--- ENTROPY ---
Probabilities: [0.33 0.33 0.33]
Entropy = 1.5834674497121084 bits

--- FINAL NEXT-WORD PREDICTION ---
Predicted word: improves

============================================================
```

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

Q1. Penn Treebank POS Tags

Sentence 1

Book an appointment with the doctor.

Word	POS Tag	Explanation
Book	VB	Verb
an	DT	Determiner
appointment	NN	Noun
with	IN	Preposition
the	DT	Determiner
doctor	NN	Noun
.	.	Punctuation

Therefore:

Here, **Book** is a verb because it represents an action:

"Book an appointment."

Sentence 2

The book contains medical information.

Word	POS Tag	Explanation
The	DT	Determiner
book	NN	Noun
contains	VBZ	Verb
medical	JJ	Adjective
information	NN	Noun
.	.	Punctuation

Therefore:

Here, **book** is a noun because it refers to a physical/informational object.

Q2. HMM Probability Calculation

Given:

Start probabilities:

The HMM score is:

For VB

For NN

Comparison:

Therefore:

Interpretation

The HMM favors **VB** because both:

1. The start probability of VB is higher.
2. The emission probability of book given VB is higher.

Thus:

is greater than:

Q3. Rule-Based vs HMM POS Tagging

Feature	Rule-Based POS Tagging	HMM/Stochastic Tagging
Method	Uses manually defined rules	Uses probabilities
Context	Uses grammatical rules	Uses transition/emission probabilities
Training	Little/no statistical training	Requires training data
Ambiguity	Can resolve using explicit rules	Resolves using probability
Flexibility	Less flexible	More flexible
Accuracy	Depends on quality of rules	Depends on training corpus
Healthcare chatbot	Useful for known patterns	Useful for varied patient language

Rule-based example

If "book" occurs after "the":

book → NN

and:

If "book" occurs at the beginning as an instruction:

book → VB

Advantages of Rule-Based Tagging

- Easy to understand
- Easy to debug
- Works well for known grammatical patterns
- Does not require a large training corpus

Limitations

- Requires many manually written rules
- Difficult to handle unusual sentences
- Rules may conflict
- Difficult to scale

Advantages of HMM Tagging

- Handles ambiguity probabilistically
- Uses context
- Can learn patterns from data
- More suitable for varied natural language

Limitations

- Requires training data
- Probability estimates can be poor with sparse data
- May make errors on unseen patterns

For a healthcare chatbot, a **hybrid approach** can be useful.

Q4. Importance of Penn Treebank POS Tagset

The Penn Treebank Tagset provides standardized tags such as:

- NN → noun
- VB → verb
- VBZ → third-person singular verb
- JJ → adjective

- DT → determiner
- IN → preposition

This standardization helps downstream NLP tasks.

1. Intent Detection

Consider:

"Book an appointment."

Book/VB indicates an action.

The chatbot can identify the intent as:

Appointment Booking

2. Entity Identification

For:

"appointment with the doctor"

nouns such as appointment and doctor can help identify important concepts/entities.

3. Response Generation

The POS structure helps the chatbot understand whether the user is:

- requesting something
- asking a question
- describing something
- providing information

4. Ambiguity Resolution

The same word can have different tags:

Book an appointment → Book/VB

The book contains... → book/NN

Therefore, POS tagging improves contextual understanding.

Q5. Python Program – Case Study 2

```
import nltk

# Download required NLTK resources
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger_eng')

# ------------------------------------------------
# SENTENCES
# ------------------------------------------------

sentences = [
    "Book an appointment with the doctor.",
    "The book contains medical information."
]

print("=" * 65)
print("AI HOSPITAL APPOINTMENT CHATBOT - POS TAGGING")
print("=" * 65)

# ------------------------------------------------
# TOKENIZATION AND POS TAGGING
# ------------------------------------------------

for i, sentence in enumerate(sentences, start=1):

    print("\nSentence", i)
    print("-" * 50)
    print(sentence)
```

```
    tokens = nltk.word_tokenize(sentence)

    print("\nTokens:")
    print(tokens)

    tags = nltk.pos_tag(tokens)

    print("\nPOS Tags:")
    for word, tag in tags:
        print(f"{word:15} -> {tag}")

# ---------------------------------------------------
# HMM CALCULATION
# ---------------------------------------------------

print("\n" + "=" * 65)
print("HMM-BASED CALCULATION FOR 'BOOK'")
print("=" * 65)

# Given probabilities

P_book_VB = 0.7
P_book_NN = 0.3

P_start_VB = 0.6
P_start_NN = 0.4

# HMM scores

score_VB = P_start_VB * P_book_VB
score_NN = P_start_NN * P_book_NN

print("\nFor VB:")
print("P(Start -> VB) =", P_start_VB)
print("P(book | VB)   =", P_book_VB)
print("Score = 0.6 × 0.7 =", score_VB)

print("\nFor NN:")
print("P(Start -> NN) =", P_start_NN)
print("P(book | NN)   =", P_book_NN)
print("Score = 0.4 × 0.3 =", score_NN)

# Most probable tag

if score_VB > score_NN:
    best_tag = "VB"
else:
    best_tag = "NN"

print("\n--- FINAL RESULT ---")
print("Probability for VB =", score_VB)
print("Probability for NN =", score_NN)
print("Most probable tag =", best_tag)

print("\n" + "=" * 65)
```

```
================================================================
AI HOSPITAL APPOINTMENT CHATBOT - POS TAGGING
================================================================

Sentence 1
--------------------------------------------------
Book an appointment with the doctor.

Tokens:
['Book', 'an', 'appointment', 'with', 'the', 'doctor', '.']

POS Tags:
Book            -> NNP
an              -> DT
appointment     -> NN
with            -> IN
the             -> DT
doctor          -> NN
.               -> .

Sentence 2
--------------------------------------------------
The book contains medical information.

Tokens:
['The', 'book', 'contains', 'medical', 'information', '.']

POS Tags:
The             -> DT
book            -> NN
contains        -> VBZ
medical         -> JJ
information     -> NN
.               -> .

================================================================
HMM-BASED CALCULATION FOR 'BOOK'
================================================================

For VB:
P(Start -> VB) = 0.6
P(book | VB)   = 0.7
Score = 0.6 × 0.7 = 0.42

For NN:
P(Start -> NN) = 0.4
P(book | NN)   = 0.3
Score = 0.4 × 0.3 = 0.12

--- FINAL RESULT ---
Probability for VB = 0.42
Probability for NN = 0.12
Most probable tag = VB

================================
```

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

Sentence:

Market growth drives investment.

Initial tags:

market/NN

growth/NN

drives/NNS

investment/NN

Transformation rule:

Change NNS to VBZ if the preceding word is tagged as NN.

Q1. Apply Transformation Rule

Initial sequence:

market/NN

growth/NN

drives/NNS

investment/NN

Look at drives/NNS.

The preceding word is:

growth/NN

The rule says:

NNS → VBZ

if preceding word = NN

Since:

drives/NNS

preceding word = growth/NN

the rule applies.

Therefore:

drives/NNS → drives/VBZ

Corrected sequence

Why is this linguistically appropriate?

The sentence is:

Market growth drives investment.

The subject is:

Market growth

This is a singular noun phrase.

Therefore, the verb must be third-person singular:

drives

Hence:

drives/VBZ

is correct.

Q2. Why is "drives" VBZ?

The initial system incorrectly assigns:

drives/NNS

NNS means **plural noun**.
However, in:
Market growth drives investment.
drives is performing an action.
It is the main verb of the sentence.
The subject is:
Market growth
Since this is singular, the verb takes the third-person singular form:
drives
Therefore:
Surrounding context
The grammatical structure is:
Market growth → Subject
drives → Verb
investment → Object
Therefore:
NN + VBZ + NN
is the appropriate structure.

Q3. Relative Frequency of Each Word
Given:

Word	Frequency
market	500
growth	350
drives	180
investment	420

Total frequency:
Relative frequency is:
Market
Growth
Drives
Investment
Distribution

Word	Frequency	Probability
market	500	0.3448
growth	350	0.2414
drives	180	0.1241
investment	420	0.2897

Check:
Importance
Frequency information helps NLP systems estimate how likely a word is.
A statistical POS tagger can combine:

- word frequency
- surrounding words
- transition probabilities
- emission probabilities

to select the most likely POS tag.
For example, although drives may sometimes appear as a noun, the surrounding grammatical context makes VBZ more likely here.

Q4. Entropy Before and After Transformation
The important point here is that **the transformation changes the POS tag, not the word frequencies**.
Therefore, the given word-frequency distribution remains:

Entropy is:
Therefore:
This gives approximately:
Before transformation
After transformation
Because the transformation only changes:
drives/NNS → drives/VBZ
and does **not change the word-frequency counts**, the word-frequency distribution remains the same.
Therefore:
Important interpretation
The entropy of the **word-frequency distribution** does not change because the frequencies did not change.
However, the **tagging uncertainty** is reduced because the incorrect tag NNS is corrected to the grammatically appropriate VBZ.
So:

- Word-frequency entropy → unchanged
- POS-tagging correctness → improved
- Linguistic uncertainty → reduced

This distinction is important in the exam answer.

Q5. Python Program – Case Study 3

```
import nltk
      corrected_tags[i] = (current_word, "VBZ")

print("\n--- TRANSFORMATION RULE ---")
print("Rule: Change NNS to VBZ if preceding word is NN")

print("\n--- CORRECTED POS TAGS ---")

for word, tag in corrected_tags:
   print(f"{word:15} -> {tag}")

# ------------------------------------------------
# WORD FREQUENCIES
# ------------------------------------------------

frequencies = {
   "market": 500,
   "growth": 350,
   "drives": 180,
   "investment": 420
}

total_frequency = sum(frequencies.values())

print("\n--- WORD FREQUENCIES ---")
print("Total frequency =", total_frequency)

probabilities = {}

for word, frequency in frequencies.items():

   probability = frequency / total_frequency
   probabilities[word] = probability

   print(
      f"{word:15} "
```

```
        f"Frequency = {frequency:4} "
        f"Probability = {probability:.4f}"
    )

# ----------------------------------------------------
# ENTROPY
# ----------------------------------------------------

entropy_before = 0

for probability in probabilities.values():
    entropy_before -= probability * math.log2(probability)

# Transformation changes POS tags only.
# Word frequencies remain unchanged.

entropy_after = entropy_before

print("\n--- ENTROPY ANALYSIS ---")

print(
    "Entropy before transformation =",
    round(entropy_before, 4),
    "bits"
)

print(
    "Entropy after transformation  =",
    round(entropy_after, 4),
    "bits"
)

# ----------------------------------------------------
# FINAL RESULT
# ----------------------------------------------------

print("\n--- FINAL POS TAGGING RESULT ---")

for word, tag in corrected_tags:
    print(f"{word:15} -> {tag}")

print("\n" + "=" * 70)
```

```
--- TRANSFORMATION RULE ---
Rule: Change NNS to VBZ if preceding word is NN

--- CORRECTED POS TAGS ---
The             -> DT
book            -> NN
contains        -> VBZ
medical         -> JJ
information     -> NN
.               -> .

--- WORD FREQUENCIES ---
Total frequency = 1450
market          Frequency =  500 Probability = 0.3448
growth          Frequency =  350 Probability = 0.2414
drives          Frequency =  180 Probability = 0.1241
investment      Frequency =  420 Probability = 0.2897

--- ENTROPY ANALYSIS ---
Entropy before transformation = 1.9161 bits
Entropy after transformation  = 1.9161 bits

--- FINAL POS TAGGING RESULT ---
The             -> DT
book            -> NN
contains        -> VBZ
medical         -> JJ
information     -> NN
.               -> .

======================================================================
```